المملكة المغربية
ⵜⴰⴳⵍⴷⵉⵜ ⵏ ⵍⵎⵖⵔⵉⴱ
وزارة التربية الوطنية
والتعليم الأولي والرياضة
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵓⵙⴳⵎⵉ ⴰⵏⴰⵎⵓⵔ
ⴷ ⴰⵙⵙⵍⵎⴷ ⴰⵎⵣⵡⴰⵔⵓ ⴷ ⵜⵓⵏⵏⵓⵏⵜ

Examen National du Brevet de Technicien Supérieur (BTS)

Session 2026

- Sujet 3 -

Filière :	**Développement des Applications Informatiques -DAI-**	**Durée :** 2 heures
Épreuve :	**Développement Mobile Android**	**Coefficient :** 45

CONSIGNES

- Le sujet comporte 4 dossiers.
- Il sera tenu compte de la qualité de rédaction.
- Aucun document n'est autorisé.

BARÈME DE NOTATION

Dossier	Intitulé	Points
1	Écosystème Mobile & Concepts	6
2	Interface XML (RelativeLayout)	4
3	Gestion des événements (Click, LongClick, StackView, CheckBox)	6
4	Validation & TextWatcher (avec gestion d'exception)	4
5	Adapter (ArrayAdapter simple et personnalisé)	6
6	Fichiers (Lecture, Modification, Suppression, Sauvegarde)	6
7	Architecture MVVM & Observation multiple (LiveData)	8
Total		**40**

Dossier 1 : Écosystème Mobile et Concepts (6 pts)

1. Expliquer brièvement le rôle de :
   - Android
   - iOS
2. Comparer les approches de développement suivantes :
   - Native
   - Web
   - Hybride

   (Technologies, avantages, inconvénients)
3. Citer les principales méthodes du cycle de vie d'une activité.
4. Expliquer la différence entre `onPause()` et `onStop()` avec un exemple réel.

Contexte général

On souhaite développer une application Android nommée **UserManager** permettant de gérer une liste d'utilisateurs.

Chaque utilisateur possède :

— un nom (String)
— un âge (int)
— un état (abonné ou non)

L'application doit permettre :
— l'ajout d'un utilisateur
— l'affichage dans un StackView
— la suppression par long clic
— la sauvegarde et la lecture des données
— la gestion via l'architecture MVVM

Dossier 2 : Interface et XML (4 pts)

1. Écrire le fichier `activity_main.xml` utilisant **RelativeLayout** contenant :
   — EditText : `etNom`
   — EditText : `etAge`
   — CheckBox : `checkAbonne`
   — Button : `btnAjouter`
   — StackView : `stackView`
2. Donner le rôle de chaque composant à travers des commentaires dans le code XML.

Dossier 3 : Adapter (6 pts)

1. Initialiser un `ArrayAdapter` simple pour afficher la liste.
   Prototype :

   ```
   public void initialiserAdapter()
   ```

2. Créer un adapter personnalisé permettant d'afficher :
   — le nom
   — l'âge
   — l'état (abonné)
   Prototype :

   ```
   public class UserAdapter extends ArrayAdapter<User>
   ```

3. Redéfinir la méthode :

   ```
   public View getView(int position, View convertView, ViewGroup parent)
   ```

Dossier 4 : Gestion des événements et accès aux données (6 pts)

Dans la classe `MainActivity`, on dispose des éléments suivants :

```
EditText etNom, etAge;
CheckBox checkAbonne;
Button btnAjouter;
StackView stackView;
ArrayList<String> liste;
ArrayAdapter<String> adapter;
```

On suppose que la classe `UserDAO` (accès aux données SQLite) est déjà implémentée.

1. Déclarer et initialiser l'objet DAO dans `MainActivity`.

   Prototype attendu :

   ```
   UserDAO dao;
   ```

2. Expliquer dans un commentaire la différence entre :
   — `getWritableDatabase()`
   — `getReadableDatabase()`
3. Écrire la méthode suivante permettant d'ajouter un utilisateur dans :
   — la base de données (via DAO)
   — la liste affichée

   ```
   public void ajouterUtilisateur()
   ```

4. Écrire le code de gestion du clic sur un élément du StackView :

   ```
   public void modifierUtilisateur(int position)
   ```

5. Écrire le code permettant de supprimer un élément avec un long clic :
   — suppression dans la base (DAO)
   — suppression dans la liste

   ```
   public void supprimerUtilisateur(int position)
   ```

Dossier 5 : Validation et TextWatcher (4 pts)

1. Implémenter un `TextWatcher` sur `etAge` permettant de vérifier que la valeur saisie est un entier valide.

   Prototype attendu :

   ```
   public void verifierAge()
   ```

2. Gérer les exceptions en affichant une erreur sur le champ.
3. Expliquer dans un commentaire ce qui se passe si l'exception n'est pas gérée.

Dossier 6 : Fichiers et persistance (6 pts)

On souhaite sauvegarder une liste d'objets `User` dans un fichier interne.
Chaque objet `User` contient :
— nom (String)
— age (int)
— abonne (boolean)

1. Écrire la méthode de sauvegarde d'une liste d'utilisateurs :

   ```
   public void sauvegarderDonnees(ArrayList<User> liste)
   ```

   (utiliser `ObjectOutputStream`)

2. Écrire la méthode de lecture :

   ```
   public ArrayList<User> lireDonnees()
   ```

   (utiliser `ObjectInputStream`)

3. Écrire une méthode permettant de modifier un utilisateur dans la liste puis sauvegarder les données :

   ```
   public void modifierUtilisateur(ArrayList<User> liste, User ancien, User nouveau)
   ```

Remarque : identifier l'objet à modifier (par exemple via le nom ou un id).

4. Écrire une méthode permettant de supprimer un utilisateur de la liste puis sauvegarder les données :

```
public void supprimerUtilisateur(ArrayList<User> liste, User u)
```

5. Écrire une méthode permettant de supprimer le fichier :

```
public void supprimerFichier()
```

6. Ajouter des commentaires expliquant :
   — pourquoi la classe `User` doit implémenter `Serializable`
   — l'intérêt de fermer les flux
   — le comportement si le fichier n'existe pas

Dossier 7 : Architecture MVVM (8 pts)

On souhaite restructurer l'application selon le modèle MVVM.

1. Créer la classe `User` (Model).
2. Créer un ViewModel contenant :

```
MutableLiveData<List<User>> users;
MutableLiveData<String> messageErreur;
MutableLiveData<Boolean> isLoading;
```

3. Implémenter les méthodes :

```
public void ajouterUser(User u)
public void supprimerUser(int position)
```

4. Dans l'activité, observer les données :

```
users.observe(...)
messageErreur.observe(...)
isLoading.observe(...)
```

5. Expliquer à travers des commentaires :
   — pourquoi on utilise LiveData
   — l'intérêt de séparer View et ViewModel

Dossier 8 : Analyse et réflexion (6 pts)

1. Expliquer pourquoi les traitements lourds doivent être évités dans le Main Thread.
2. Donner un exemple concret de blocage d'interface.
3. Expliquer l'impact d'une mauvaise gestion du cycle de vie sur :
   — les données
   — l'interface
4. Qu'est-ce qu'un `Bundle` en Android ? Donner son rôle et un exemple d'utilisation dans une activité.